

Rust 핵심 문법 및 명령어 총정리

1. Rust 전체 처리 흐름 요약

Rust는 컴파일(빌드)할 때 프로그램의 메모리 안전성을 미리 검사하여 런타임(실행 중) 오류와 데이터 충돌을 완전히 예방하는 언어다.

- **의존성 및 환경 설정 (Cargo.toml)**: 프로젝트 정보와 사용할 외부 라이브러리 목록을 적어두는 곳이다.
- **데이터 구조 및 기능 설계 (struct, enum, trait)**: 로봇의 데이터와 상태를 종류별로 묶고, 장치들의 공통 동작 규칙을 정한다.
- **컴파일러 안전 검사 (Borrow Checker)**: 소유권이 잘 이동했는지, 빌려 쓰는 규칙을 잘 지켰는지 검사하여 메모리 실수를 빌드할 때 잡아낸다.
- **프로젝트 가동 및 에러 처리 (main, match)**: 시작점인 `main()` 함수가 실행된 후, 문제가 생길 수 있는 코드를 안전하게 나누어 처리한다.

2. 핵심 문법 및 코드 정리

① 기본 구조, 변수, 타입, 출력, 함수

프로그램의 시작점과 기본적인 데이터 저장 및 반환 방식이다.

```
fn main() { // Rust 프로그램이 가장 먼저 실행하는 시작점
    // 1. 변수 선언과 값 수정 제어 (기본적으로는 한 번 넣은 값을 바꿀 수 없음)
    let linear_velocity: f64 = 0.5; // let: 변수 만들기. f64: 실수형 데이터
    let mut battery_level: i32 = 100; // mut: 값을 바꿀 수 있는 변수로 만들기
    battery_level = 95; // 새로운 값 저장

    // 2. 데이터 종류 바꾸기 (as)
    // Rust는 규칙이 엄격해서 정수(i32)와 실수(f64)를 그냥 계산하면 에러가 남
    let float_battery = battery_level as f64;

    // 3. 화면 출력 도구
    // 이름 뒤에 '!'가 붙는 것은 일반 함수가 아니라 글자를 찍어주는 특별 매크로임
    println!("Robot Speed: {} m/s", linear_velocity); // {}: 값이 들어갈 자리
}
```

함수 값 반환 규칙: 마지막 줄에 세미콜론(`;`)을 붙이지 않으면 `return` 글자를 안 적어도 그 결과값이 함수 밖으로 나감. 세미콜론을 붙이면 반환할 값이 없어서 에러가 남.

```
fn add_speed(a: i32, b: i32) -> i32 { // -> 뒤에 반환할 데이터 종류 지정
    a + b                               // 세미콜론 없음 -> 계산 결과 내보내기
}
```

② 조건문, 반복문, 배열, 튜플, 슬라이스

실행 흐름을 조절하고 여러 데이터를 다루는 고정된 크기의 묶음 구조다.

- **엄격한 if 조건문:** `if` 옆의 조건식 결과는 반드시 참/거짓(`bool`) 형태여야 함. 다른 언어처럼 숫자 `1` 이나 `0` 을 참/거짓 대신 넣으면 에러가 남.
- **세 가지 반복문 구조:**
 - `loop`: 멈추지 않는 무한 반복문. 안에서 `break` 를 만나면 멈추고 빠져나옴.
 - `while`: 지정한 조건이 참인 동안에만 계속 반복함.
 - `for`: 정해진 횟수나 배열 안을 차례대로 돌. `0..5` 는 0부터 4까지 돌며 끝 숫자는 포함 안 함.
- **여러 값 묶기:**
 - **배열 ([값; 개수]):** 모두 같은 종류의 데이터들을 정해진 개수만큼 묶어서 보관함. 방 번호는 0번부터 시작함.
 - **튜플 ((값, 값, 값)):** 서로 다른 종류의 데이터들을 소괄호로 묶음. 점(`.`)과 방 번호로 접근함.
- **슬라이스 (Slice):** 원본 데이터를 복사하지 않고, 배열의 특정 일부분만 안전하게 빌려서 가리키는 기능임.

```
// 조건식, 반복 범위 및 슬라이스 활용 예시
if value > 0 { println!("정상"); } // 무조건 참/거짓이 나오는 비교식 작성

for i in 1..4 { ... } // 끝 숫자 4를 제외한 1, 2, 3 순회

let lidar_array = [10, 20, 30, 40];
let part: &[i32] = &lidar_array[1..3]; // 1번 방부터 2번 방까지 빌려오기 (&[20, 30])
```

③ 소유권(Ownership), 이동(Move), 복사(Copy), 참조(Borrowing)

메모리를 안전하게 관리하기 위해 값 하나당 주인 변수를 딱 하나만 지정하는 규칙이다.

- **소유권 규칙:** 모든 값은 주인이 있으며, 주인 변수가 속한 중괄호(`{ }`) 블록이 끝나면 값도 메모리에서 자동으로 지워짐.

• 이동 (Move) vs 복사 (Copy):

- **이동 (Move)**: 덩치가 큰 타입(`String`, `Vec`)을 새 변수에 대입하거나 함수에 넘기면 **물건의 주인이 바뀜**. 주인을 잃은 이전 변수를 다시 쓰려고 하면 에러가 남. 똑같은 데이터를 하나 더 새로 복제해서 유지하고 싶을 때만 `clone()` 을 씀.
- **복사 (Copy)**: 기본 숫자나 참/거짓 타입(`i32`, `f64`, `bool`)은 대입할 때 값이 그냥 복사되므로 기존 변수도 그대로 쓸 수 있음.

• 빌려 쓰기 (Borrowing):

주인을 바꾸지 않고 참조 기호(`&`)를 붙여서 잠깐 빌려 쓰는 방식임.

- **`&T`** (불변 참조): 읽기만 가능하게 빌림. 여러 곳에서 동시에 빌려 갈 수 있음.
- **`&mut T`** (가변 참조): 값을 고칠 수 있게 빌림. 변수 값 앞에 별표(`*`)를 붙여서 실제 데이터를 수정함.
- **안전 규정**: 동시에 값을 고치다가 프로그램이 꼬이는 걸 막기 위해, 한 번에 딱 한 곳만 고치게 빌려줄 수 있음. 또 한 누군가 값을 읽고(불변 참조) 있는 동안에는 값을 고치게(가변 참조) 빌려줄 수 없음.

```
// 값을 고치게 빌려주고 실제 데이터를 수정하는 코드 예시
let mut speed = 0.5;
let r = &mut speed; // 고칠 수 있는 권한을 r 변수에 빌려줌
*r = 1.0;           // 별표(*)를 사용해 원본 speed의 실제 값을 1.0으로 변경
```

④ 구조체(Struct), 열거형(Enum), 매치(Match), Option, Result

연관된 데이터를 묶고, 여러 상황이나 실패 가능성을 안전하게 나누어 처리하는 도구다.

```
struct Robot { name: String, mode: RobotMode } // 구조체: 이름, 상태 등 여러 데이터를 하나로 묶기

enum RobotMode { Idle, Moving, Error } // 열거형: 정해진 몇 가지 상태 선택지 정의

impl Robot { // impl: 구조체에 사용할 함수(기능)들을 모아 두는 곳
    fn check_mode(&self) { // &self: 내 로봇 데이터를 읽기 전용으로 가져옴
        // match: 열거형에 있는 모든 상태를 빠짐없이 처리하도록 컴파일러가 강제함
        match self.mode { // 하나라도 조건 처리를 빼먹으면 에러가 남
            RobotMode::Idle => println!("대기"),
            RobotMode::Moving => println!("이동"),
            RobotMode::Error => println!("에러 발생!"),
        }
    }
}
```

비어있거나 실패할 수 있는 상황 처리: Rust는 프로그램이 갑자기 꺼지게 만드는 빈 값(null)이 없음.

- `Option<T>` : 값이 들어있을 수도 있고(`Some(값)`), 텅 비어있을 수도 있음(`None`)을 표현함.
- `Result<T, E>` : 연산이 성공하면 결과물(`Ok(성공값)`)을 주고, 실패하면 원인(`Err(에러내용)`)을 반환함.

⑤ 동적 배열(Vec), 문자열(String), 트레이트(Trait), 임플(Impl), 모듈, Cargo

개수가 계속 변하는 데이터를 다루고 프로젝트를 만들어 빌드하는 단계다.

```
// 트레이트(Trait): 여러 구조체들이 공통으로 가져야 할 동작 규칙을 약속함
trait Sensor {
    fn read_data(&self) -> Result, String;
}

struct Lidar { topic_name: String } // String: 글자를 추가하고 고칠 수 있는 문자열 타입

// impl 트레이트명 for 구조체명: 약속한 공통 규칙대로 기능을 구현함
impl Sensor for Lidar {
    fn read_data(&self) -> Result, String> {
        // Vec<T>: 개수가 자유롭게 늘어나는 동적 배열 (LIDAR 센서 여러 거리값 저장에 적합)
        let mut ranges = vec![1.2, 1.5, 2.0]; // vec!: 동적 배열을 쉽게 만드는 매크로
        ranges.push(0.8); // 값 추가하기. 값을 변경하므로 변수 선언에
mut 필수

        Ok(ranges)
    }
}
```

- **모듈 불러오기 (`use`)**: 다른 파일에 있는 코드나 도구 라이브러리를 가져와서 사용함.

Cargo 명령어	실제 수행하는 작업
<code>cargo new <이름></code>	새로운 Rust 프로젝트 폴더 구조를 알아서 만들어줌
<code>cargo check</code>	실제 빌드는 안 하고 문법이나 타입 에러가 있는지만 초고속으로 검사함
<code>cargo build</code>	전체 코드를 컴파일(해석)해서 실행 가능한 파일로 만들어줌
<code>cargo run</code>	코드를 빌드한 후 만들어진 파일을 즉시 실행까지 해줌

* 프로젝트 이름, 버전, 사용할 외부 라이브러리 목록 정보는 `Cargo.toml` 파일에서 관리함.

3. 실전 트러블슈팅 및 디버깅 프로세스

컴파일러가 에러 메시지를 띄우며 빌드를 막았을 때 문제를 찾아 해결하는 순서다.

1. **빌려 쓰기 범위 확인**: 값을 고치고 읽는 참조가 꼬여서 에러가 나면, 중괄호(`{ }`)를 사용해 고치는 코드의 범위를 좁혀주어 사용이 끝나면 자원이 바로 풀리게 만듦.
2. **함수 인자의 소유권 체크**: 데이터를 읽기만 하는 함수에 원본 데이터를 그대로 넘겨서 주인이 바뀌었는지 확인하고, 원본을 보존하기 위해 참조 기호(`&`) 타입으로 함수 인자를 교체함.
3. **mut 권한 확인**: 배열에 값을 추가(`push()`)하거나 구조체 필드 값을 고칠 때, 처음에 변수를 선언한 곳에 `mut` 글자를 빼먹지 않았는지 확인하고 적어줌.
4. **조건 분기 누락 보완**: `match` 구문을 쓸 때 실패한 에러(`Err`)나 빈 값(`None`) 케이스 처리를 귀찮아서 생략했는지 확인하고 모든 선택지를 빠짐없이 채워줌.
5. **데이터 유효성 확인**: 동적 배열 데이터 연산을 시작하기 전, 배열의 길이(`len()`)가 0인지 사전에 체크하여 텅 비어 있을 때는 에러를 안전하게 돌려주도록 예외 구조를 만듦.